Universidad Tecnológica Nacional
Facultad Regional

TP Transacciones

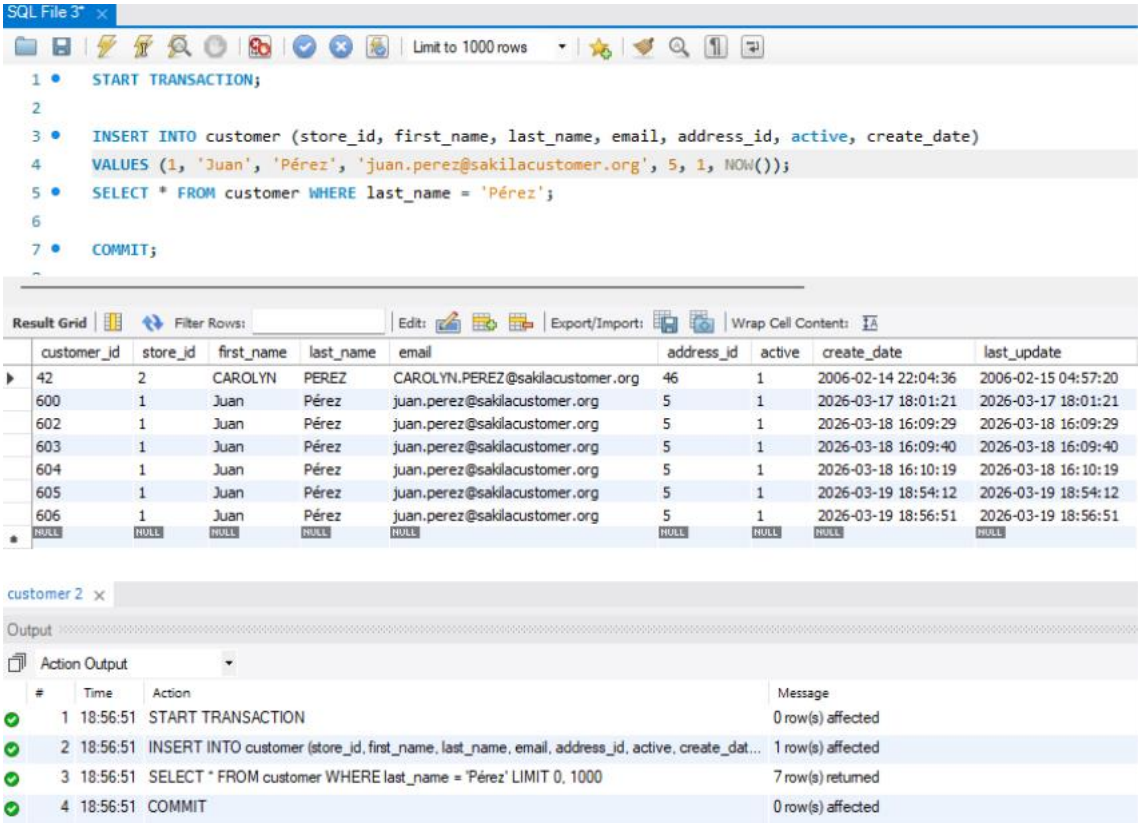
Base de Datos II

Cátedra:	Base de Datos II
Profesora:	Suligoy, Eliana
Alumno:	Baru, Matías
Creación:	17 de marzo de 2026
Entrega:	20 de marzo de 2026

ESCENARIO: 1

A. EJECUCION:

Se inició la transacción con `START TRANSACTION`, luego se insertó un nuevo cliente con apellido Pérez y se realizó una consulta dentro de la misma transacción para verificar el registro, finalizando con la confirmación de los cambios mediante `COMMIT`.



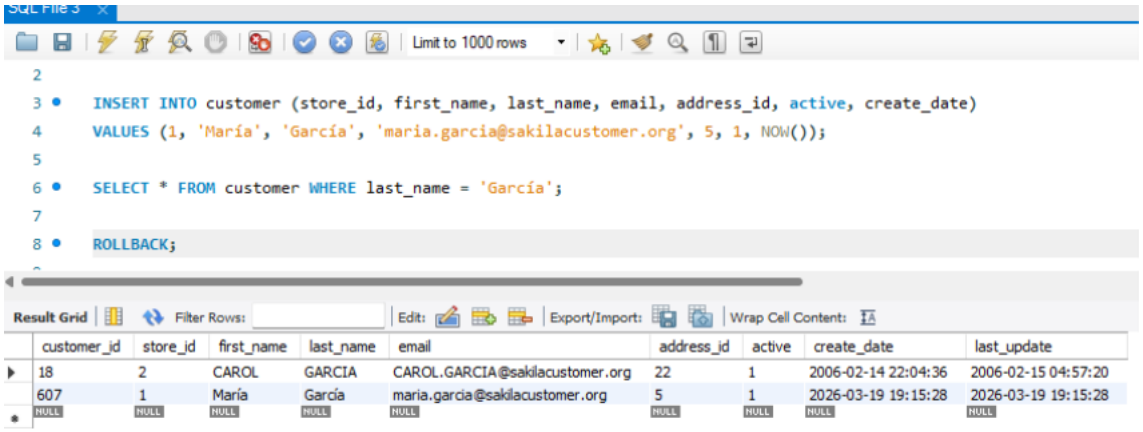
B. La transacción se confirmó exitosamente mediante COMMIT, ya que luego de ejecutar el INSERT se utilizó dicho comando para guardar los cambios de forma permanente en la base de datos. Cuando se usa START TRANSACTION, los cambios como el INSERT no se almacenan inmediatamente sino que quedan pendientes, lo que permitió verificar mediante el SELECT que el registro fue agregado correctamente dentro de la transacción antes de confirmarlo; finalmente, al ejecutar COMMIT, MySQL guardó definitivamente los cambios, quedando el nuevo cliente almacenado de forma permanente, lo cual se puede

confirmar en la captura donde aparecen varios registros con apellido "Pérez", incluido el recién insertado.

ESCENARIO: 2

A. EJECUCION:

Se inició la transacción con `START TRANSACTION`, luego se insertó un nuevo cliente llamado María García y se realizó una consulta `SELECT` para verificar que el registro había sido agregado correctamente dentro de la transacción, finalizando con la ejecución de `ROLLBACK` para deshacer todos los cambios realizados.



```
INSERT INTO customer (store_id, first_name, last_name, email, address_id, active, create_date)
VALUES (1, 'María', 'García', 'maria.garcia@sakilacustomer.org', 5, 1, NOW());

SELECT * FROM customer WHERE last_name = 'García';

ROLLBACK;
```

customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
18	2	CAROL	GARCIA	CAROL.GARCIA@sakilacustomer.org	22	1	2006-02-14 22:04:36	2006-02-15 04:57:20
607	1	María	García	maria.garcia@sakilacustomer.org	5	1	2026-03-19 19:15:28	2026-03-19 19:15:28
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

customer 3

Output

Action Output

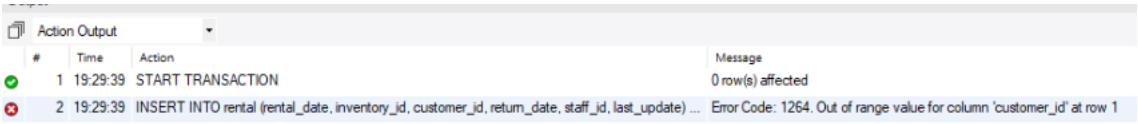
#	Time	Action	Message
3	18:56:51	SELECT * FROM customer WHERE last_name = 'Pérez' LIMIT 0, 1000	7 row(s) returned
4	18:56:51	COMMIT	0 row(s) affected
5	19:15:28	START TRANSACTION	0 row(s) affected
6	19:15:28	INSERT INTO customer (store_id, first_name, last_name, email, address_id, active, create_d...	1 row(s) affected
7	19:15:28	SELECT * FROM customer WHERE last_name = 'García' LIMIT 0, 1000	2 row(s) returned
8	19:15:28	ROLLBACK	0 row(s) affected

B. La transacción fue abortada mediante `ROLLBACK`, lo que significa que los cambios realizados por el `INSERT` no fueron guardados en la base de datos. Al utilizar `START TRANSACTION`, los cambios quedan pendientes hasta que se confirmen o deshagan; en este caso, al ejecutar `ROLLBACK`, MySQL deshizo el `INSERT` del nuevo cliente María García, por lo que el registro nunca llegó a persistirse de forma permanente. El `SELECT` realizado dentro de la transacción permitió verificar que el registro existía temporalmente, pero una vez ejecutado el `ROLLBACK`, dicho registro desapareció por completo, como si la operación nunca hubiera ocurrido.

ESCENARIO: 3

A. EJECUCION:

Se inició la transacción con `START TRANSACTION` y se intentó insertar un registro en la tabla `rental`, sin embargo se produjo un error con el código 1264 indicando que el valor ingresado para la columna `customer_id` estaba fuera del rango permitido, finalizando con la ejecución de `COMMIT`.



Action Output

#	Time	Action	Message
1	19:29:39	START TRANSACTION	0 row(s) affected
2	19:29:39	INSERT INTO rental (rental_date, inventory_id, customer_id, return_date, staff_id, last_update) ...	Error Code: 1264. Out of range value for column 'customer_id' at row 1

B. ANALISIS: La transacción no se confirmó correctamente debido a que el `INSERT` falló con el error 1264 ("Out of range value"), ya que el valor ingresado en la columna `customer_id` (999999) supera el rango permitido por el tipo de dato definido en la tabla `rental`. Al producirse este error, MySQL no insertó ningún registro, por lo que cuando se ejecutó el `COMMIT` al final, no había ningún cambio pendiente que confirmar, haciendo que dicho comando no tuviera efecto real sobre la base de datos. En pocas palabras, la operación principal falló antes de llegar al `COMMIT`, por lo que la base de datos quedó exactamente igual que al inicio, sin ninguna modificación.

ESCENARIO: 4

A. EJECUCION:

Se inició la transacción con `START TRANSACTION`, luego se insertó un nuevo registro en la tabla `rental` y se guardó el ID generado automáticamente mediante `LAST_INSERT_ID()` en la variable `@nuevo_rental_id`, el cual fue utilizado para insertar un pago relacionado en la tabla `payment`, confirmando finalmente todos los cambios de forma permanente con `COMMIT`.

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:38:27	START TRANSACTION	0 row(s) affected	0.000 sec
2	19:38:27	INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id, last_update) VALUES (N...	1 row(s) affected	0.000 sec
3	19:38:27	SET @nuevo_rental_id = LAST_INSERT_ID()	0 row(s) affected	0.000 sec
4	19:38:27	INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date) VALUES (1, 1,...	1 row(s) affected	0.000 sec
5	19:38:27	COMMIT	0 row(s) affected	0.000 sec

B. ANALISIS:

La transacción fue confirmada exitosamente mediante `COMMIT`, ya que todas las operaciones se ejecutaron sin errores. Primero se insertó un registro en la tabla `rental` y se utilizó `LAST_INSERT_ID()` para capturar el ID generado automáticamente, almacenándolo en la variable de sesión `@nuevo_rental_id`, la cual fue utilizada para insertar un registro en la tabla `payment` vinculando correctamente el pago con el alquiler recién creado. Gracias a que ambas operaciones se realizaron dentro de la misma transacción, el `COMMIT` garantizó que los dos registros se guardaran de forma permanente y consistente en la base de datos, asegurando la integridad entre las tablas `rental` y `payment`.

ESCENARIO: 5

A. EJECUCION:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

email
carlos.lopez@nuevo.com

Output

Action Output

#	Time	Action	Message
1	19:38:27	START TRANSACTION	0 row(s) affected
2	19:38:27	INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id, last_update) VALUES ...	1 row(s) affected
3	19:38:27	SET @nuevo_rental_id = LAST_INSERT_ID()	0 row(s) affected
4	19:38:27	INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date) VALUES (1...	1 row(s) affected
5	19:38:27	COMMIT	0 row(s) affected
6	19:45:22	START TRANSACTION	0 row(s) affected
7	19:45:22	UPDATE customer SET first_name = 'Carlos' WHERE customer_id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
8	19:45:22	SELECT first_name FROM customer WHERE customer_id = 3 LIMIT 0, 1000	1 row(s) returned
9	19:45:22	SAVEPOINT despues_nombre	0 row(s) affected
10	19:45:22	UPDATE customer SET last_name = 'López' WHERE customer_id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
11	19:45:22	SELECT last_name FROM customer WHERE customer_id = 3 LIMIT 0, 1000	1 row(s) returned
12	19:45:22	SAVEPOINT despues_apellido	0 row(s) affected
13	19:45:22	UPDATE customer SET email = 'carlos.lopez@nuevo.com' WHERE customer_id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
14	19:45:22	SELECT email FROM customer WHERE customer_id = 3 LIMIT 0, 1000	1 row(s) returned
15	19:45:22	ROLLBACK TO despues_apellido	0 row(s) affected
16	19:45:22	COMMIT	0 row(s) affected

B. ANALISIS:

La transacción se confirmó correctamente, ya que el log muestra que se completó exitosamente con el comando `COMMIT` al final. Esto ocurrió porque todos los pasos se ejecutaron sin errores: primero se inició la transacción con `START TRANSACTION`, luego se insertó un nuevo registro en la tabla `rental` con datos válidos como fecha, `inventory_id`, `customer_id`, `staff_id` y `last_update`, después se obtuvo el ID generado automáticamente mediante `LAST_INSERT_ID()` y se almacenó en la variable `@nuevo_rental_id`, que fue utilizada para insertar el registro correspondiente en la tabla `payment` asociando correctamente el pago con el nuevo alquiler, y finalmente el `COMMIT` confirmó que todos los cambios fueron guardados de forma permanente en la base de datos sin ningún inconveniente.

ESCENARIO: 6

A. EJECUCION:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

email
carlos.lopez@nuevo.com

customer 13 | customer 14 | customer 15

Output

Action Output

#	Time	Action	Message
1	19:59:02	START TRANSACTION	0 row(s) affected
2	19:59:02	UPDATE customer SET first_name = 'Carlos' WHERE customer_id = 3	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
3	19:59:02	SELECT first_name FROM customer WHERE customer_id = 3 LIMIT 0, 1000	1 row(s) returned
4	19:59:02	SAVEPOINT despues_nombre	0 row(s) affected
5	19:59:02	UPDATE customer SET last_name = 'López' WHERE customer_id = 3	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
6	19:59:02	SELECT last_name FROM customer WHERE customer_id = 3 LIMIT 0, 1000	1 row(s) returned
7	19:59:02	SAVEPOINT despues_apellido	0 row(s) affected
8	19:59:02	UPDATE customer SET email = 'carlos.lopez@nuevo.com' WHERE customer_id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
9	19:59:02	SELECT email FROM customer WHERE customer_id = 3 LIMIT 0, 1000	1 row(s) returned
10	19:59:02	ROLLBACK TO despues_apellido	0 row(s) affected
11	19:59:02	COMMIT	0 row(s) affected

B. ANALISIS:

La transacción no se confirmó completamente debido al uso de `ROLLBACK TO despues_apellido`, que revirtió los cambios hasta un punto intermedio antes de ejecutar el `COMMIT`. Esto ocurrió porque la transacción fue dividida en varios pasos utilizando `SAVEPOINT`: primero se inició con `START TRANSACTION`, luego se actualizó el primer nombre del cliente y se guardó un punto de control llamado `despues_nombre`, después se actualizó el apellido y se guardó otro punto llamado `despues_apellido`, y finalmente se actualizó el correo electrónico. Sin embargo, antes del `COMMIT` se ejecutó `ROLLBACK TO despues_apellido`, lo que deshizo únicamente el cambio del correo electrónico, manteniendo las actualizaciones de

nombre y apellido. Como resultado, aunque el `COMMIT` se ejecutó al final, solo se guardaron permanentemente los cambios realizados hasta el savepoint `despues_apellido`, siendo este un claro ejemplo de cómo los `SAVEPOINT` permiten controlar parcialmente una transacción sin necesidad de revertirla por completo.